1

```
int n = 0, mx = INT_MIN;

void pushm(int x){
    S.push(x);
    mx = max(mx, x);
    S.push(mx);
    n += 2;
}

int popm(){
    S.pop();
    int x = S.pop();
    n -= 2;
    mx = n ? S.top() : INT_MIN;
    return x;
}

int topm(){
    int m = S.pop();
    int x = S.top();
    S.push(m);
    return x;
}

int getMax(){
    return S.top();
}
```

每个真实元素 x 压栈后，再额外压入一个辅助整数，表示压入 x 后栈中所有真实元素的最大值。

2

考虑 n 的情况，设此时的答案为 C_n。每个合法括号序列都可以唯一地写为 $(A)B$ 的形式，其中 A, B 分别是含 $i, n-1-i$ 对括号的合法序列。

因此，枚举 i 与对应的 (A, B) 的对数即可。则有递推式

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-i-1}$$

由 $C_0 = 1$，归纳可得 C_n 即为卡特兰数

$$C_n = \frac{(2n)!}{n!(n+1)!}$$

$C_3 = 5$，这 5 种括号序列为：

1. ((()))
2. (())()
3. ()(())
4. ()()()

5. (()())

3

```
void visit(website p){
    Back.push(current);
    current = p;
    while(!Forward.empty())
        Forward.pop();
}

void back(){
    if(Back.empty()) return;
    Forward.push(current);
    current = Back.pop();
}

void forward(){
    if(Forward.empty()) return;
    Back.push(current);
    current = Forward.pop();
}
```

- 最终：`current = A`；
- `Back` 栈从栈底到栈顶：`[Home]`
- `Forward` 栈从栈底到栈顶：`[C]`

4

`S1` 保存新入栈的元素；`S2` 保存等待出队的元素，其栈顶是当前的队首。

```
void enQueue(int x){
    push(S1, x);
}

int deQueue(){
    if(!isEmpty(S2)) return pop(S2);
    while(!isEmpty(S1)) push(S2, pop(S1));
    return pop(S2);
}

bool isEmpty(){
    return isEmpty(S1) && isEmpty(S2);
}
```

单次操作最坏：入队 $O(1)$，出队 $O(n)$，判断是否为空 $O(1)$。

三次出队返回 $5, 8, 2$。

最终队列为 `[7]`。